Pythia: Speculative Dispatch for Autonomous Agent Orchestration in Scientific Computing

[Double-Blind Review — Authors Omitted]
[Affiliations Omitted]

***Abstract*—Multi-agent AI systems are becoming integral to scientific computing workflows, yet their orchestration layers — which decide what agents to dispatch, where, and with what context — remain a significant performance bottleneck. We observe that this bottleneck is structurally identical to latency problems solved by CPU speculative execution and LLM speculative decoding: dispatch decisions are predictable, and verification is cheaper than stalling. We introduce Pythia, a speculative dispatch framework that adapts the draft-target speculation paradigm to multi-agent orchestration. Pythia overlaps dispatch planning with speculative pre-execution across three progressive modes — context preparation, agent pre-dispatch, and draft execution with verification — each with formally derived break-even accuracy thresholds. A reinforcement learning component improves speculation accuracy over time by learning user-specific dispatch patterns, creating an orchestration system whose latency decreases with usage. We implement Pythia within an NSF-funded scientific computing platform and evaluate it on four HPC-adjacent workload suites. Results demonstrate [X%] dispatch latency reduction under mature speculation, with the learner achieving [Y%] speculation accuracy after [N] interactions and a wasted compute ratio below [W%] at optimal confidence thresholds.**

***Index Terms*—multi-agent orchestration, speculative execution, reinforcement learning, HPC scheduling, scientific computing, LLM agents**

I. Introduction

Scientific computing stands at an inflection point. Autonomous AI agents are no longer peripheral tools but active participants in research workflows — generating MPI code, constructing data processing pipelines, orchestrating experiments across heterogeneous infrastructure, and analyzing results at scales that exceed manual human effort. National cyberinfrastructure initiatives are integrating these agents into platforms that span GPU-accelerated clusters, multi-tier storage hierarchies, and cloud computing resources, creating distributed multi-agent systems that must coordinate dozens of specialized agents across heterogeneous hardware and multiple AI providers simultaneously.

This coordination — the *orchestration layer* — has become a critical performance bottleneck. When a researcher issues a natural-language request to a multi-agent scientific computing platform, the orchestrator must classify the intent, decompose the task, select appropriate agents, match them to available resources, assemble context from scientific datasets (HDF5, NetCDF, Zarr, and others), construct prompts, and dispatch execution — all before any productive computation begins. We observe that this dispatch overhead constitutes [X]% of end-to-end response latency for typical scientific computing tasks, and that the problem compounds in heterogeneous environments where the dispatch solver must navigate provider-specific constraints: API rate limits, token budgets, GPU memory availability, and cost ceilings across multiple AI providers and compute tiers.

Yet dispatch decisions are not random. A computational scientist who spends mornings on code review and afternoons on data analysis exhibits strong temporal regularity. A materials science pipeline that repeatedly invokes the same sequence of agents for structure optimization, property prediction, and visualization follows a predictable dispatch pattern. We find that for the top-K intent classes — which cover [Z]% of all requests in our evaluation — the dispatch plan has low conditional entropy given the classified intent, making prediction tractable.

This combination of bottleneck and predictability is structurally identical to latency problems that have been solved in two domains well known to the HPC community.

In processor microarchitecture, *speculative execution* predicts branch outcomes and executes instructions along the predicted path before the branch is resolved; if correct, results commit at zero additional cost [1]. Modern TAGE-class predictors achieve accuracy exceeding 95% [2]. In large language model inference, *speculative decoding* uses a small draft model to generate candidate tokens that a larger target model verifies in a single forward pass, achieving 2–3$\times$ throughput improvements [3], [4]. Both paradigms share a principle that generalizes: *when verification is cheaper than stalling, and prediction accuracy improves with observation, speculative execution amortizes latency*.

We introduce **Pythia**, a speculative dispatch framework that transfers the draft-target speculation paradigm to multi-agent orchestration for scientific computing. When a request arrives, Pythia's lightweight speculative dispatcher predicts the likely dispatch plan and immediately begins pre-execution — context assembly, agent warm-up, resource reservation, and even draft task execution — while the full dispatch solver computes the optimal plan in parallel. The solver's output either *commits* the speculative work, *partially commits* it, or *flushes* the speculation entirely. A reinforcement learning component — the *Learner* — observes dispatch outcomes and continuously refines prediction accuracy, creating an orchestration system whose latency *decreases the more it is used* by a given researcher on a given infrastructure.

We formalize speculation as a three-mode progressive hierarchy. **Mode 1** (Speculative Context Preparation) prefetches

context and warms caches — the dispatch analog of hardware cache prefetching — with near-zero misprediction cost. **Mode 2** (Speculative Agent Pre-dispatch) predicts which agents are needed and begins provisioning — the dispatch analog of speculative execution past a branch — gated by a learned confidence threshold. **Mode 3** (Speculative Execution with Verification) deploys a fast draft agent that begins producing output verified by the target agent upon arrival — the direct analog of LLM speculative decoding applied to multi-agent task execution. For each mode, we derive a formal cost model establishing the break-even speculation accuracy — the same cost structure that governs branch prediction profitability, now applied to agent dispatch.

This paper makes the following contributions:

1) **The Speculative Dispatch abstraction.** A formal framework adapting the draft-target speculation paradigm from CPU architecture and LLM inference to multi-agent orchestration, including a three-mode speculation hierarchy and a commit/partial-commit/flush reconciliation protocol.
2) **A learner-augmented speculation model.** A reinforcement learning system that improves speculation accuracy over time by learning user-specific dispatch patterns, with progressive mode activation as the natural exploration-exploitation tradeoff.
3) **A resource-aware dispatch solver.** An optimization engine that treats heterogeneous infrastructure constraints as first-class scheduling parameters, connecting multi-agent dispatch to the HPC scheduling literature.
4) **Implementation and evaluation.** A prototype integrated into an NSF-funded scientific computing platform, evaluated on four HPC-adjacent workload suites.

Section II provides background on speculative execution and characterizes the dispatch bottleneck. Section III presents the Pythia architecture, speculation modes, and formal cost model. Section IV describes the Learner's reinforcement learning formulation. Section V details the implementation. Section VI presents the evaluation. Section VII discusses limitations and future directions. Section VIII concludes.

II. Background and Motivation

This section establishes the technical foundations for speculative dispatch by reviewing speculation in CPU architecture and LLM inference, surveying the multi-agent orchestration landscape, and presenting empirical evidence that dispatch planning is both a bottleneck and predictable.

A. Speculative Execution in CPU Architecture

Modern out-of-order processors routinely execute instructions before all dependencies are resolved. When the processor encounters a conditional branch, rather than stalling the pipeline, a branch predictor predicts the outcome and the processor speculatively executes along the predicted path [1]. If correct, speculative results commit; if wrong, the work is flushed, incurring a penalty of 10–20 cycles on modern microarchitectures.

Branch prediction has evolved through four generations. Static heuristics gave way to two-level adaptive predictors [5], then to tournament predictors combining multiple strategies [6], and finally to TAGE predictors using tagged geometric history lengths that achieve accuracy exceeding 95% [2], [7]. Neural branch prediction introduced perceptrons as an alternative, enabling linear scaling with history length [8].

The cost model is fundamental: speculation is net-positive when prediction accuracy exceeds $\frac{C_{flush}}{L_{saved}+C_{flush}}$, where C_{flush} is the misprediction penalty and L_{saved} is the latency avoided. For typical pipeline depths, the break-even accuracy is approximately 70–75% — well below modern predictor performance.

The security implications are equally instructive. Spectre demonstrated that speculative memory accesses, even when architecturally rolled back, leave observable traces that can leak data across security boundaries [9]. This lesson transfers directly: speculative resource access at the dispatch layer may similarly leak context in multi-tenant deployments (Section VII).

B. Speculative Decoding in LLM Inference

Speculative decoding adapts the draft-verify paradigm to autoregressive language model inference. A small draft model generates γ candidate tokens, then the larger target model scores all candidates in a single forward pass — exploiting the asymmetry between generation ($O(\gamma)$ passes) and verification ($O(1)$ pass) [3], [4].

A modified rejection sampling scheme ensures the output distribution is identical to the target model alone [3]. The field has diversified: SpecInfer uses tree-based speculation with model ensembles [10]; Medusa adds lightweight MLP heads for self-drafting [11]; EAGLE conditions drafting on the target's hidden states [12], [13]; and online speculative decoding continuously updates the draft from observed queries [14] — a direct precedent for our Learner's online adaptation.

Speedup is governed by the acceptance rate α, speculation depth γ, and cost ratio c; practical systems achieve 2–3$\times$ throughput with α in the range 0.6–0.85 [15].

C. Multi-Agent Orchestration

Multi-agent LLM systems assign specialized roles to agents that collaborate through structured communication. AutoGen organizes conversable agent groups [16]; MetaGPT encodes standardized procedures into assembly-line workflows [17]; ChatDev decomposes software development into chat chains [18]. Patterns such as ReAct interleave reasoning with tool use [19], while Reflexion adds verbal self-reflection [20].

All existing frameworks follow the same sequential pipeline: intent recognition $\rightarrow$ task decomposition $\rightarrow$ agent assignment $\rightarrow$ execution. Recent work on LLM routing [21] and mixture-of-agents [22] improves selection quality but remains reactive — classify *then* route, with no speculation, no pre-execution, and no learning from dispatch history.

TABLE I
SPECULATION PARADIGM ACROSS THREE DOMAINS.

	CPU	LLM Decoding	Pythia (Ours)
Draft	Branch predictor	Small draft model	Dispatch predictor
Target	Branch resolution	Large target model	Full dispatch solver
Unit	Instructions	Tokens	Dispatch plans
Commit	Correct prediction	Token accepted	Plan matches
Flush	Misprediction	Token rejected	Plan mismatch
Cost	Flush cycles	Draft compute	Agent init waste
Learning	TAGE [2]	Online SD [14]	RL Learner
Break-even	∼70–75%	∼50–60%	Per mode (§III-D)
Security	Spectre [9]	N/A	§VII

The gap is clear: the multi-agent orchestration layer lacks the predictive, speculative, and adaptive capabilities that have proven transformative in CPU architecture and LLM inference.

D. Empirical Motivation

[Profiling methodology: instrument the dispatch pipeline of a production multi-agent scientific computing platform handling HPC-adjacent workloads over N requests.]

[Table or figure: dispatch latency breakdown by stage — intent classification, task decomposition, agent selection, context assembly, agent initialization. Show dispatch overhead as % of end-to-end latency.]

[Predictability analysis: conditional entropy of dispatch plans given intent class for top-K intent classes covering Z% of requests.]

The profiling reveals two findings: (1) dispatch overhead is a substantial fraction of end-to-end latency, particularly for shorter tasks; and (2) dispatch decisions exhibit strong temporal locality — recurring patterns map to consistent plans, making prediction tractable.

E. Positioning

Table I summarizes the structural parallel across domains.

The draft-target-verify-commit/flush pattern is domain-independent. It applies wherever (a) an expensive optimization can be approximated cheaply, (b) verification is cheaper than computing the exact solution, and (c) prediction accuracy improves with observation. Multi-agent dispatch satisfies all three conditions. To our knowledge, this is the first work to formalize and evaluate this transfer.

III. PYTHIA: SPECULATIVE DISPATCH

This section presents the Pythia framework: the five-layer architecture, three progressive speculation modes, reconciliation protocol, formal cost model, and resource-aware dispatch formulation.

[Architecture diagram: 5-layer pipeline. Request → Intent Detector → parallel split to Solver & Speculative Dispatcher → Orchestrator reconciliation → Agent Execution → Learner feedback loop.]

Fig. 1. Pythia's five-layer speculative dispatch architecture. The Speculative Dispatcher races ahead of the Solver, and the Orchestrator reconciles their outputs.

A. Architecture Overview

The Pythia pipeline consists of five components with well-defined data contracts (Figure 1).

Intent Detector. A lightweight classifier that transforms a natural-language request into a structured intent: task type, complexity estimate, domain tags, decomposability score, and constraint annotations. The Intent Detector is deliberately shallow — fast classification, not planning.

Dispatch Solver (Target). The full optimization engine computing the optimal dispatch plan given complete system state: all intents, the agent pool, fleet configuration, resource availability, and cost constraints. Solver latency is the bottleneck that speculation aims to hide.

Speculative Dispatcher (Draft). A lightweight prediction model running in parallel with the Solver, producing a draft plan using intent pattern matching, a historical dispatch cache, and a learned user fingerprint maintained by the Learner.

Orchestrator (Reconciliation Engine). Receives both plans, compares element-by-element, and issues COMMIT, PARTIAL COMMIT, or FLUSH before executing the final dispatch.

Learner. A reinforcement learning component observing the full dispatch lifecycle and continuously updating the Speculative Dispatcher (Section IV).

B. Speculation Modes

We define three progressive modes, each subsumes the previous with higher reward and greater misprediction cost.

Mode 1: Speculative Context Preparation. *Analogy: CPU cache prefetching.* Upon intent classification, Pythia fetches relevant context, warms tool configurations, and pre-assembles prompt templates. This work is useful regardless of the final plan — context is agent-agnostic. Let $C_{prep}^{waste} \approx 0$; Mode 1 is activated unconditionally.

Mode 2: Speculative Agent Pre-dispatch. *Analogy: speculative execution past a branch.* Pythia predicts which agents are needed and begins provisioning: allocating resources, initializing runtimes, establishing connections. Gated by confidence threshold τ_2. The expected cost is $(1-p) \cdot C_{init}$ and expected benefit is $p \cdot L_{init}$; Mode 2 is profitable when:

$$p > \frac{C_{init}}{L_{init} + C_{init}} = \tau_2^* \tag{1}$$

This is structurally identical to the break-even condition in CPU branch prediction.

Mode 3: Speculative Execution with Verification. *Analogy: LLM speculative decoding.* A fast draft agent begins producing task output while the target agent is provisioned with the Solver's optimal plan. If aligned, draft output is accepted; if not, discarded. Mode 3 is profitable when the output acceptance probability q exceeds:

$$q > \frac{C_{draft} + C_{flush}^{M3}}{L_{target} + C_{flush}^{M3}} = \tau_3^* \tag{2}$$

mirroring the acceptance rate threshold in speculative decoding.

C. Reconciliation Protocol

When the Solver's optimal plan P^* arrives and speculative plan $\hat{P}$ is in progress:

COMMIT ($P^* = \hat{P}$): All pre-executed work accepted. Zero additional dispatch latency.

PARTIAL COMMIT ($P^* \cap \hat{P} \neq \emptyset$, $P^* \neq \hat{P}$): Correctly speculated work retained; mismatched agents redirected. The *salvage ratio* $\sigma = |P^* \cap \hat{P}|/|\hat{P}|$ governs the cost: $(1-\sigma) \cdot C_{redirect}$.

FLUSH ($P^* \cap \hat{P} = \emptyset$): All speculation discarded. Solver's plan executes from scratch.

D. Cost Model

Let L_s be Solver latency, p_c, p_{pc}, p_f be COMMIT, PARTIAL COMMIT, and FLUSH probabilities ($p_c + p_{pc} + p_f = 1$), and $\bar{\sigma}$ be the mean salvage ratio.

Baseline latency: $L_{baseline} = L_s + L_{exec}$

Speculative latency:

$$L_{pythia} = L_s + p_{pc}(1-\bar{\sigma})L_{redirect} + p_f \cdot L_{exec} \tag{3}$$

Latency reduction:

$$\Delta L = L_{exec} - [p_{pc}(1-\bar{\sigma})L_{redirect} + p_f \cdot L_{exec}] \tag{4}$$

Speculation is net-positive when $\Delta L > 0$: the weighted probability of usable pre-execution exceeds the relative cost of wasted work.

Wasted compute ratio:

$$W = \frac{p_f \cdot C_{spec} + p_{pc}(1-\bar{\sigma}) \cdot C_{spec}}{C_{total}} \tag{5}$$

directly comparable to the wasted instruction ratio in CPU speculation.

E. Resource-Aware Dispatch

The Solver operates over a heterogeneous fleet $\mathcal{F} = \{f_1, \ldots, f_n\}$ with capability vectors $f_i = (\text{compute}_i, \text{memory}_i, \text{rate_limit}_i, \text{token_budget}_i, \text{cost_rate}_i, \text{latency}_i)$, optimizing subject to capacity, budget, rate-limit, and affinity constraints. This formulation connects directly to HPC scheduling [23], [24], extending classical constraint satisfaction with AI-specific dimensions — token budgets and API rate limits — that have no analog in traditional job scheduling.

IV. Learner-Augmented Speculation

The Learner transforms Pythia from static prediction into an adaptive system.

A. Reinforcement Learning Formulation

We formulate the speculation policy as a contextual bandit.

State: $s_t = (\mathbf{i}_t, \mathbf{f}_t, \mathbf{h}_t)$ — intent features, fleet state, and dispatch fingerprint (compressed recent history).

Action: $a_t = \hat{P}_t$ — the speculative dispatch plan, including mode selection and resource allocations.

Reward:

$$r_t = \begin{cases} +L_{saved} & \text{COMMIT} \\ +\sigma_t L_{saved} - (1-\sigma_t)C_{redirect} & \text{PARTIAL} \\ -C_{flush} & \text{FLUSH} \end{cases} \tag{6}$$

The Learner maintains policy $\pi_\theta(a|s)$ maximizing $J(\theta) = \mathbb{E}[r(s,a)]$. The confidence score maps directly to mode selection: above τ_3^* activates Mode 3; above τ_2^* activates Mode 2; otherwise Mode 1.

The dispatch fingerprint $\mathbf{h}_t$ encodes the k most recent (intent, plan, outcome) triples via a small recurrent network, capturing temporal patterns in a researcher's workflow.

B. Progressive Activation

The Learner governs staged activation mirroring CPU predictor evolution:

Cold start ($< N_1$ interactions): Mode 1 only. Pure observation. Optional Bayesian priors from aggregate patterns.

Early learning (N_1–N_2): High-frequency intent classes exceed τ_2^*, enabling Mode 2 for predictable requests. Analogous to two-level adaptive prediction [5].

Mature operation ($> N_2$): Top-k intent classes reach τ_3^* accuracy, enabling Mode 3. Analogous to TAGE-class neural prediction [2].

The thresholds N_1, N_2 are emergent — they depend on workload predictability, not fixed hyperparameters.

C. Convergence and Adaptation

For the contextual bandit with finite intent classes, speculation accuracy converges at $O(\sqrt{T \log |\mathcal{A}|/T})$ [25]. Convergence is fast in practice because effective action spaces are small — most researchers invoke a handful of intent patterns repeatedly.

Non-stationarity (concept drift) is handled via: (1) windowed history with natural decay of stale patterns, and (2) drift detection triggering mode regression and accelerated relearning when accuracy drops below τ_2^*.

V. Implementation

A. Prototype

We implement Pythia within Clio Coder, the multi-agent orchestration component of an NSF-funded scientific computing platform. The platform deploys autonomous AI agents across heterogeneous infrastructure — GPU-accelerated nodes, cloud instances, and multi-tier storage hierarchies — to enable natural-language interaction with scientific datasets and HPC

workflows. Agents access 14+ scientific data formats through a Model Context Protocol (MCP) server ecosystem.

[Describe the non-speculative baseline dispatch pipeline in Clio Coder — how requests currently flow through intent parsing, planning, agent selection, context assembly, and execution.]

Pythia adds four components. **[Intent Detector implementation: model type, latency, input/output schema.] [Speculative Dispatcher: prediction model (lookup table, neural net, or small LLM), confidence scoring.] [Reconciliation Engine: COMMIT/PARTIAL/FLUSH logic, state salvage for partial commits.] [Learner: RL framework, policy network architecture, online vs. batch updates, fingerprint encoding.]**

Components communicate via structured contracts: `Intent`, `DispatchPlan`, `SpeculationResult`, `ReconciliationOutcome`, and `DispatchOutcome`.

B. Deployment

[Infrastructure: local nodes, cloud instances, HPC cluster. AI providers and rate limits. Fleet capability vectors.]

Data-format awareness enables domain-specific speculation signals — HDF5 requests trigger different dispatch patterns than source code analysis — improving prediction for scientific workflows.

VI. Evaluation

We evaluate Pythia across four workloads, four baselines, and seven metrics, answering: (1) Does speculation reduce dispatch latency? (2) Does the Learner improve accuracy? (3) What is the misprediction cost? (4) Parameter sensitivity? (5) Scalability?

A. Experimental Setup

Workloads: (1) HPC Code Generation — MPI, Slurm scripts, parallel debugging; (2) Scientific Data Pipelines — HDF5/NetCDF processing, format conversion; (3) Research Workflow Automation — end-to-end literature-to-analysis workflows; (4) Dispatch Micro-benchmarks — isolated mode characterization.

Baselines: No Speculation (sequential pipeline), Static Heuristic (rule-based), Speculation without Learning (fixed heuristics), Oracle (perfect prediction upper bound).

Metrics: Dispatch latency, speculation hit rate, wasted compute ratio, dispatch quality, learner convergence, cost efficiency, scalability.

[Hardware/software specs. Repeat counts. Statistical methodology.]

B. Dispatch Latency

[Primary result figure: dispatch latency across workloads $\times$ baselines $\times$ modes. Mode 1 provides modest, consistent reduction. Mode 2 substantial for predictable intents. Mode 3 maximum reduction. Improvement grows with workload predictability.]

C. Speculation Accuracy

[Learning curves: hit rate over interactions per workload. Cold start $\rightarrow$ Mode 2 activation $\rightarrow$ Mode 3 activation. SwoL ceiling comparison. Regular patterns converge faster.]

D. Cost Analysis

[Pareto frontier: wasted compute vs. confidence threshold per mode. Mode 1 near-zero waste. Optimal thresholds. Net cost efficiency despite mispredictions.]

E. Ablation Studies

[Table: mode combinations, threshold sensitivity, history window size, single vs. multi-user, homogeneous vs. heterogeneous infrastructure.]

F. Scalability

[Fleet size, agent diversity, search space complexity. Solver latency increases with fleet size; speculation benefit increases proportionally.]

VII. Discussion

A. Limitations

Cold start. The Learner requires history to move beyond Mode 1. Mode 2 activation requires approximately **[N_1]** interactions for common intents. Bayesian priors from aggregate behavior could reduce this, but we have not evaluated this approach.

Unpredictable workloads. Speculative dispatch assumes temporal locality in dispatch decisions. For exploratory tasks with high dispatch entropy, the system correctly regresses to Mode 1 via drift detection, limiting benefit to context preparation.

Single-user scope. Our evaluation focuses on single-user scenarios. Multi-user deployments introduce resource contention for speculative pre-execution — extending the framework with fair-share speculative scheduling is future work.

B. Security Implications

Speculative dispatch raises concerns directly analogous to Spectre [9]. Speculative context prefetching may access resources based on a *predicted* plan differing from the *authorized* optimal plan, creating a potential information channel in multi-tenant deployments. We identify three mitigations: speculative isolation, deferred context binding, and speculative access auditing — but defer full analysis to future work.

C. Future Directions

HPC scheduler integration. The Solver's constraint formulation maps naturally to Slurm's resource model, enabling agent orchestration across managed cluster resources. **Federated learning.** Cross-user dispatch patterns could reduce cold-start times with privacy guarantees. **Transfer learning.** Preserving generalizable patterns across project transitions while rapidly adapting to new dispatch requirements.

VIII. Conclusion

Multi-agent AI systems are increasingly central to scientific computing, yet their orchestration layers remain a performance bottleneck. This paper introduced *Pythia*, a speculative dispatch framework adapting the draft-target paradigm — proven in CPU branch prediction and LLM speculative decoding — to multi-agent orchestration.

We formalized a three-mode speculation hierarchy with cost models deriving break-even accuracy thresholds for each mode, a learner-augmented speculation model that improves dispatch prediction through reinforcement learning, and a resource-aware solver connecting multi-agent dispatch to HPC scheduling. Our prototype, integrated into an NSF-funded scientific computing platform, demonstrates **[X%]** dispatch latency reduction and learner convergence within **[N]** interactions.

The convergence of HPC systems expertise and AI agent orchestration presents an opportunity for the scientific computing community. The scheduling, resource management, and performance optimization principles that underpin decades of HPC research apply directly to the emerging challenge of orchestrating autonomous agents across heterogeneous infrastructure. Pythia demonstrates one instance of this transfer; we anticipate that further HPC-inspired optimizations will prove equally productive as multi-agent systems scale.

References

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA: Morgan Kaufmann, 2017.

[2] A. Seznec, "A new case for the TAGE branch predictor," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2011, pp. 117–127.

[3] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 19 274–19 286.

[4] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[5] T.-Y. Yeh and Y. N. Patt, "Two-level adaptive training branch prediction," in *Proceedings of the 24th Annual International Symposium on Microarchitecture (MICRO)*. ACM, 1991, pp. 51–61.

[6] S. McFarling, "Combining branch predictors," Digital Equipment Corporation Western Research Laboratory, Palo Alto, CA, Tech. Rep. TN-36, 1993.

[7] A. Seznec and P. Michaud, "A case for (partially) TAgged GEometric history length branch prediction," *Journal of Instruction-Level Parallelism*, vol. 8, pp. 1–23, 2006.

[8] D. A. Jimenez and C. Lin, "Dynamic branch prediction with perceptrons," in *Proceedings of the 7th International Symposium on High-Performance Computer Architecture (HPCA)*, 2001, pp. 197–206.

[9] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, "Spectre attacks: Exploiting speculative execution," in *Proceedings of the 40th IEEE Symposium on Security and Privacy (S&P)*, 2019, pp. 1–19.

[10] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, "SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[11] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 5209–5235.

[12] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*. PMLR, 2024.

[13] ——, "EAGLE-2: Faster inference of language models with dynamic draft trees," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[14] X. Liu, L. Hu, P. Bailis, A. Cheung, Z. Deng, I. Stoica, and H. Zhang, "Online speculative decoding," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 31 131–31 146.

[15] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui, "Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 7655–7671.

[16] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[17] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[18] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 15 174–15 186.

[19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[20] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[21] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," *arXiv preprint arXiv:2406.18665*, 2024.

[22] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, "Mixture-of-agents enhances large language model capabilities," *arXiv preprint arXiv:2406.04692*, 2024.

[23] D. G. Feitelson and L. Rudolph, "Parallel job scheduling: Issues and approaches," in *Job Scheduling Strategies for Parallel Processing (JSSPP)*, ser. LNCS, vol. 949. Springer, 1995.

[24] A. B. Yoo, M. A. Jette, and M. Grondona, "SLURM: Simple Linux utility for resource management," in *Job Scheduling Strategies for Parallel Processing (JSSPP)*, ser. LNCS, vol. 2862. Springer, 2003, pp. 44–60.

[25] A. Agarwal, D. Hsu, S. Kale, J. Langford, L. Li, and R. E. Schapire, "Taming the monster: A fast and simple algorithm for contextual bandits," in *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 2014.